VI ĐIỀU KHIỂN

WIFI MODULE

Hoàng Đăng Khôi	22T_Nhat2
Thân Ánh Thiện	22T_Nhat2
Phạm Quang Thắng	23T_KHDL2
Trần Hồ Đình Tuấn	22T_DT4

01

Khái niệm

02

Cấu tạo

03

Nguyên lí hoạt động

04

Ứng dụng

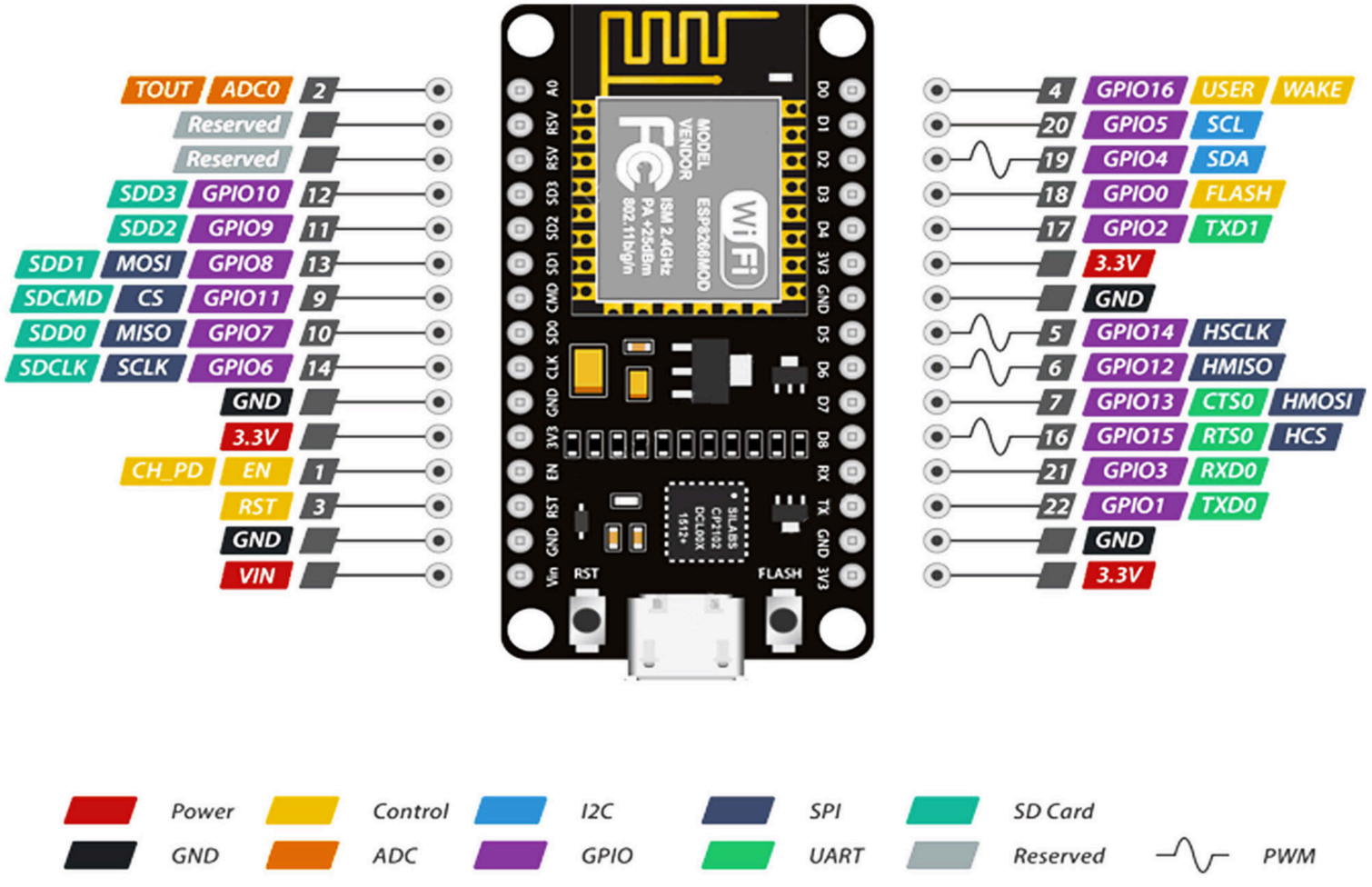
01 Khái niệm

Module wifi

Module WiFi ESP8266 là thiết bị dùng để kết nối WiFi và truyền dữ liệu giữa các thiết bị qua giao thức TCP/IP. ESP8266 có thể hoạt động ở 3 chế độ: AP (Access Point), STA (Station) hoặc AP+STA, phù hợp cho các ứng dụng IoT như nhà thông minh và điều khiển từ xa.

02 Cấu tạo

Module wifi



-**Chân kết nối:** Bao gồm các chân cấp nguồn (VCC, GND), chân TX/RX cho giao tiếp UART, và các chân GPIO để điều khiển thiết bị ngoại vi hoặc giao tiếp với các cảm biến..

-Anten: Giúp cải thiện tầm hoạt động và chất lượng kết nối WiFi của module.

-LED chỉ trạng thái: Để chỉ trạng thái hoạt động của module.

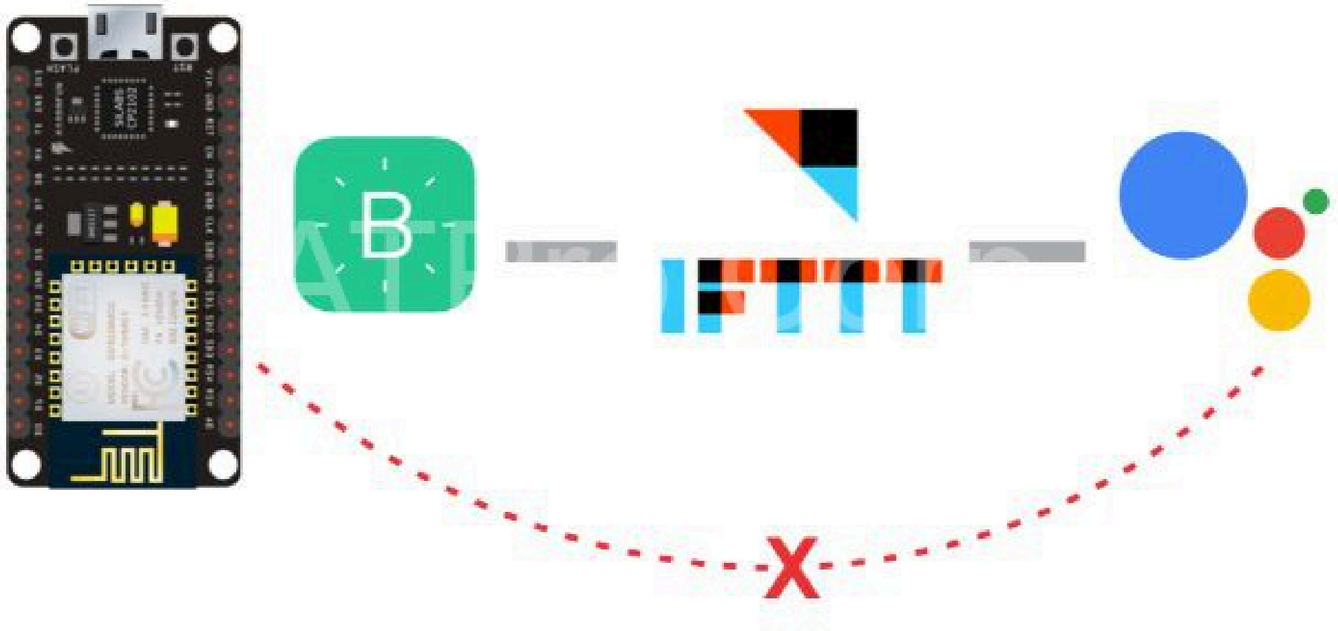
-**Dải chuyển đổi nguồn điện:** Đảm bảo cung cấp nguồn điện ổn định cho module, thông thường là 3.3V, và có thể được kết nối trực tiếp vào nguồn điện từ pin hoặc nguồn 5V qua bộ chuyển đổi.

03 Nguyên lý hoạt động

Module ESP8266 hoạt động dựa trên giao tiếp dữ liệu nối tiếp thông qua hai chân UART, trong đó chân TX giao tiếp với RX của vi điều khiển và RX giao tiếp với TX của vi điều khiển.

Module có hai chế độ hoạt động:

- Chế độ dữ liệu (Data Mode), khi không có tín hiệu đặc biệt, module sẽ truyền/nhận dữ liệu giữa thiết bị và mạng WiFi
- Chế độ dòng lệnh (Command Mode), khi cấp tín hiệu logic vào chân điều khiển hoặc cấu hình qua phần mềm, cho phép gửi các lệnh AT để cấu hình các thông số mạng, địa chỉ IP hoặc các chức năng khác.



04 Ứng dụng

4.1 Demo webserver điều khiển động cơ và hiển thị kết quả cảm biến realtime

Lắp mạch

Cảm biến	Module WiFi
DO	D5
VCC	3V3
GND	GND

Động cơ bước	Module Wifi
IN1	D4
IN2	D3
IN3	D2
IN4	D1
*Các chân còn lại tham khảo bài học về động cơ bước đã học trước đó	

Code

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <Stepper.h>

// Replace with your network credentials
const char *ssid = "Dang Khoa2";
const char *password = "dangkhoi";


#define lineSensor D5
#define IN1 D4
#define IN2 D3
#define IN3 D2
#define IN4 D1
```

- Khai báo các thư viện cần thiết

- ESP8266.h để quản lý các tính năng liên quan đến wifi cho module
- ESPAsyncTCP.h để hỗ trợ giao thức bất đồng bộ giữa server và client
- ESPAsyncWebServer.h để hỗ trợ việc giao tiếp giữa web và thiết bị bằng giao thức bất đồng bộ

- Khai báo tên wifi, password wifi. Nên nhập đúng tên mật khẩu, cách viết hoa, thường của wifi mà bạn muốn module kết nối đến

- Định nghĩa các chân mà bạn muốn động cơ, cảm biến kết nối đến module wifi

Code

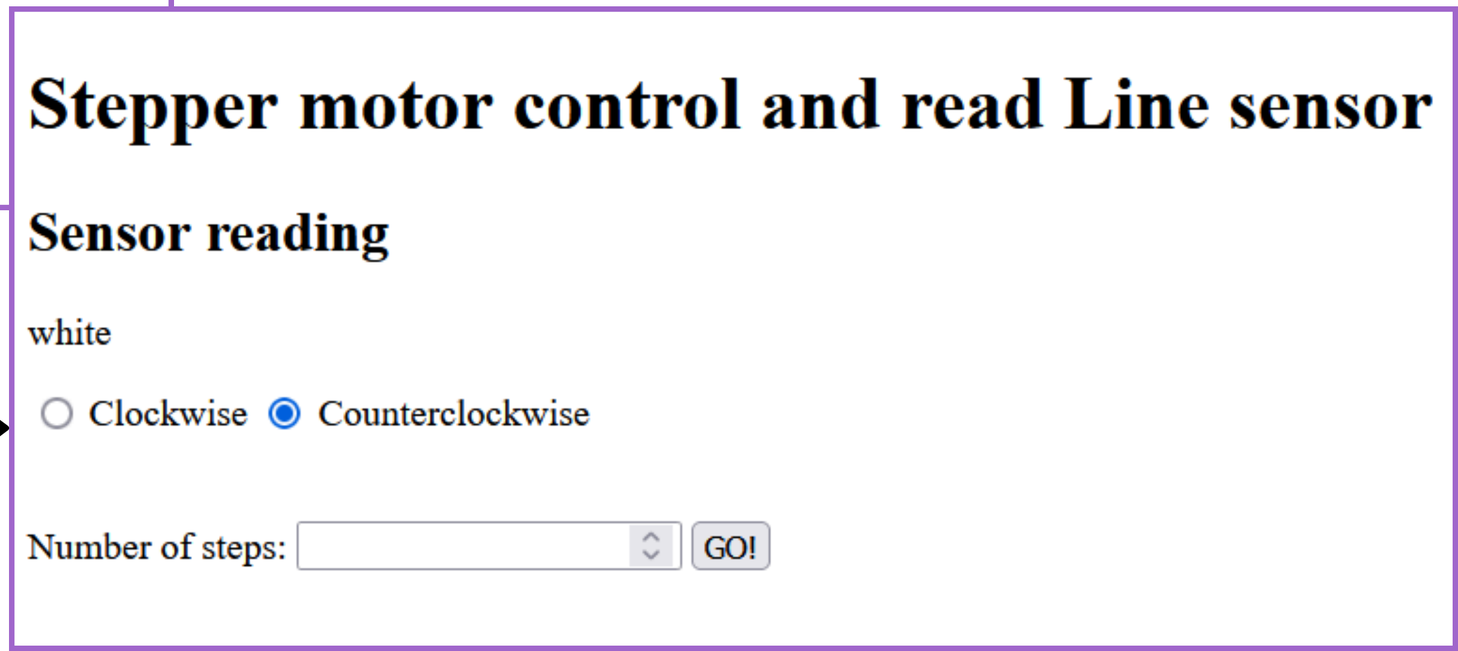
```
const int stepPerRevolution = 2048;
Stepper myStepper(stepPerRevolution, IN1, IN3, IN2, IN4);

// Create AsyncWebServer object on port 80
AsyncWebServer server(80);

// Variables to save values from HTML form
const char *PARAM_INPUT_1 = "direction";
const char *PARAM_INPUT_2 = "steps";
String direction;
String steps;

unsigned long lastServoUpdate = 0;
unsigned long servoDelay = 1000;

String readLineSensor() {
  return (digitalRead(lineSensor) == HIGH) ? "white" : "black";
}


// Variable to detect whether a new request occurred
bool newRequest = false;
```

- Thiết lập các thông tin cần thiết cho động cơ mà bạn muốn điều khiển
- Thực hiện tạo webserver qua cổng 80. Ở đây bạn có thể thực hiện qua cổng khác nhưng khi truy cập web cần dùng URL có định dạng IP:tên cổng
- Định nghĩa các biến sẽ nhận diện dữ liệu cần thiết từ web để điều khiển động cơ
- Định nghĩa các biến lưu dữ liệu cần thiết từ web trả về để điều khiển động cơ ở phía server
- Định nghĩa biến ghi nhớ thời điểm gần nhất thực hiện chạy động cơ
- Định nghĩa biến delay cho động cơ
- Định nghĩa hàm để đọc kết quả từ cảm biến rồi cho ra kết quả hiển thị trên web
- Tạo biến nhận biết có request từ web

Code

```
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <title>Stepper Motor & Sensor</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
<body>
  <h1>Stepper motor control and read Line sensor</h1>
  <h2>Sensor reading</h2>
  <p><span id ="line">%LINE%</span></p>
  <form id="motorForm" >
      <input type="radio" name="direction" value="CW" checked>
      <label for="CW">Clockwise</label>
      <input type="radio" name="direction" value="CCW">
      <label for="CW">Counterclockwise</label><br><br><br>
      <label for="steps">Number of steps:</label>
      <input type="number" name="steps">
      <input type="submit" value="GO!">
    </form>
</body>
```

- Khai báo một biến hằng PROGMEM để lưu trữ nội dung HTML cho trang web, biến này sẽ lưu trữ trong bộ nhớ flash của module nhằm tiết kiệm dung lượng cho RAM



Code

```
61 <script>
62     document.getElementById('motorForm').addEventListener('submit', function (e) {
63       e.preventDefault(); // Ngăn form tải lại trang
64
65       const formData = new FormData(this); // Lấy dữ liệu từ form
66
67       fetch("/", {
68           method: "POST",
69           body: formData
70       })
71       .then(response => response.text()) // Chuyển đổi phản hồi từ server thành văn bản
72       .catch(error => console.error("Error:", error)); // Xử lý lỗi
73   });
```

- Khai báo hàm để xử lý dữ liệu trong form trước khi gửi đến cho server thông qua phương thức POST, hàm này sẽ không buộc trang phải tải lại.

Code

```
74   setInterval(function(){
75     var xhttp=new XMLHttpRequest();
76     xhttp.onreadystatechange = function(){
77       if(this.readyState == 4 && this.status == 200){
78         document.getElementById("line").innerHTML = this.responseText;
79       }
80     };
81     xhttp.open("GET", "/line", true);
82     xhttp.send();
83   }, 5000);
84 </script>
85 </html>
86 )rawliteral";
```

- Khai báo hàm tự động cập nhật dữ liệu cảm biến trên web sau mỗi 5s.
- Dòng 75 tạo một đối tượng XMLHttpRequest() để xử lý request bất đồng bộ
- Dòng 76->80, khi request đã được hoàn tất và gửi đến client thành công, thay đổi nội dung của thành phần có ID “line” thành văn bản phản hồi từ phía server.
- Dòng 81, định nghĩa request này là phương thức GET - phương thức nhận dữ liệu từ server lên client, tại endpoint “/line”, true tức đây là request được xử lý bất đồng bộ
- Dòng 82 gửi request đi.

Code

```
// Function to replace placeholders in HTML
String processor(const String &var)
{
  if (var == "LINE")
  {
    return readLineSensor();
  }
  return String();
}
```

```
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <title>Stepper Motor & Sensor</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
</head>
<body>
  <h1>Stepper motor control and read Line sensor</h1>
  <h2>Sensor reading</h2>
  <p><span id="line">%LINE%</span></p>
  <form id="motorForm">
      <input type="radio" name="direction" value="CW" checked>
      <label for="CW">Clockwise</label>
      <input type="radio" name="direction" value="CCW">
      <label for="CW">Counterclockwise</label><br><br><br>
      <label for="steps">Number of steps:</label>
      <input type="number" name="steps">
      <input type="submit" value="GO!">
    </form>
</body>
```

- - Khai báo hàm processor để thay thế biến LINE trên web thành kết quả của hàm readLineSensor

Code

```
void initWiFi()
{
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting to WiFi ..");
  while (WiFi.status() != WL_CONNECTED)
  {
    Serial.print('.');
    delay(1000);
  }
  Serial.println(WiFi.localIP());
}
```

- Khai báo hàm initWiFi() để xử lý việc kết nối wifi cho module.
- Thiết lập chế độ station cho module, với chế độ này module sẽ trở thành thiết bị có thể kết nối đến wifi
- Truyền biến tên wifi, mật khẩu wifi vào hàm begin() để bắt đầu việc kết nối wifi cho module
- Khi kết nối đã thành công, in IP trong serial monitor. Với IP này bạn sẽ truy cập được vào web.

Code

```
void setup()
{
  myStepper.setSpeed(5);
  Serial.begin(9600);
  initWiFi();
  pinMode(lineSensor, INPUT);
```

- Bắt đầu hàm setup(), set up cho động cơ, cảm biến, kết nối wifi, giao tiếp serial

Code

```
119   // Web Server Root URL
120   server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)
121             { request->send_P(200, "text/html", index_html, processor); });
122
```

```
88 // Function to replace placeholders in HTML
89 String processor(const String &var)
90 {
91   if (var == "LINE")
92   {
93     return readLineSensor();
94   }
95   return String();
96 }
```

- Hàm server.on() thứ nhất xử lý request GET khi truy cập đường dẫn vào trang web bằng hàm lambda như trên
- Hàm lambda này sẽ thực hiện việc gửi biến chứa nội dung HTML trang và nội dung trả về từ hàm processor khi request này OK (200)

Code

```
// Web Server Root URL
server.on("/", HTTP_GET, [](AsyncWebServerRequest *request)
          { request->send_P(200, "text/html", index_html, processor); });

server.on("/line", HTTP_GET, [](AsyncWebServerRequest *request){
  request->send_P(200, "text/plain", readLineSensor().c_str());
});

```

```
setInterval(function(){
  var xhttp=new XMLHttpRequest();
  xhttp.onreadystatechange = function(){
    if(this.readyState == 4 && this.status == 200){
      document.getElementById("line").innerHTML = this.responseText;
    }
  };
  xhttp.open("GET", "/line", true);
  xhttp.send();
}, 5000);
```

- Hàm server.on() tiếp theo để xử lý request GET, gửi nội dung kết quả của hàm readLineSensor() đến endpoint "/line"
- Hàm này giúp cập nhật dữ liệu tại endpoint "/line" thay vì toàn trang, tránh tải lại cả trang để cập nhật biến %LINE%

Code

```
server.on("/", HTTP_POST, [](AsyncWebServerRequest *request)
          {
  int params = request->params();
  for(int i=0;i<params;i++){
    AsyncWebParameter* p = request->getParam(i);
    if(p->isPost()){
      // HTTP POST input1 value (direction)
      if (p->name() == PARAM_INPUT_1) {
        direction = p->value().c_str();
        Serial.print("Direction set to: ");
        Serial.println(direction);
      }
      // HTTP POST input2 value (steps)
      if (p->name() == PARAM_INPUT_2) {
        steps = p->value().c_str();
        Serial.print("Number of steps set to: ");
        Serial.println(steps);
      }
    }
  }
  request->send(200, "text/html", index_html);
  newRequest = true; });

  server.begin();
```

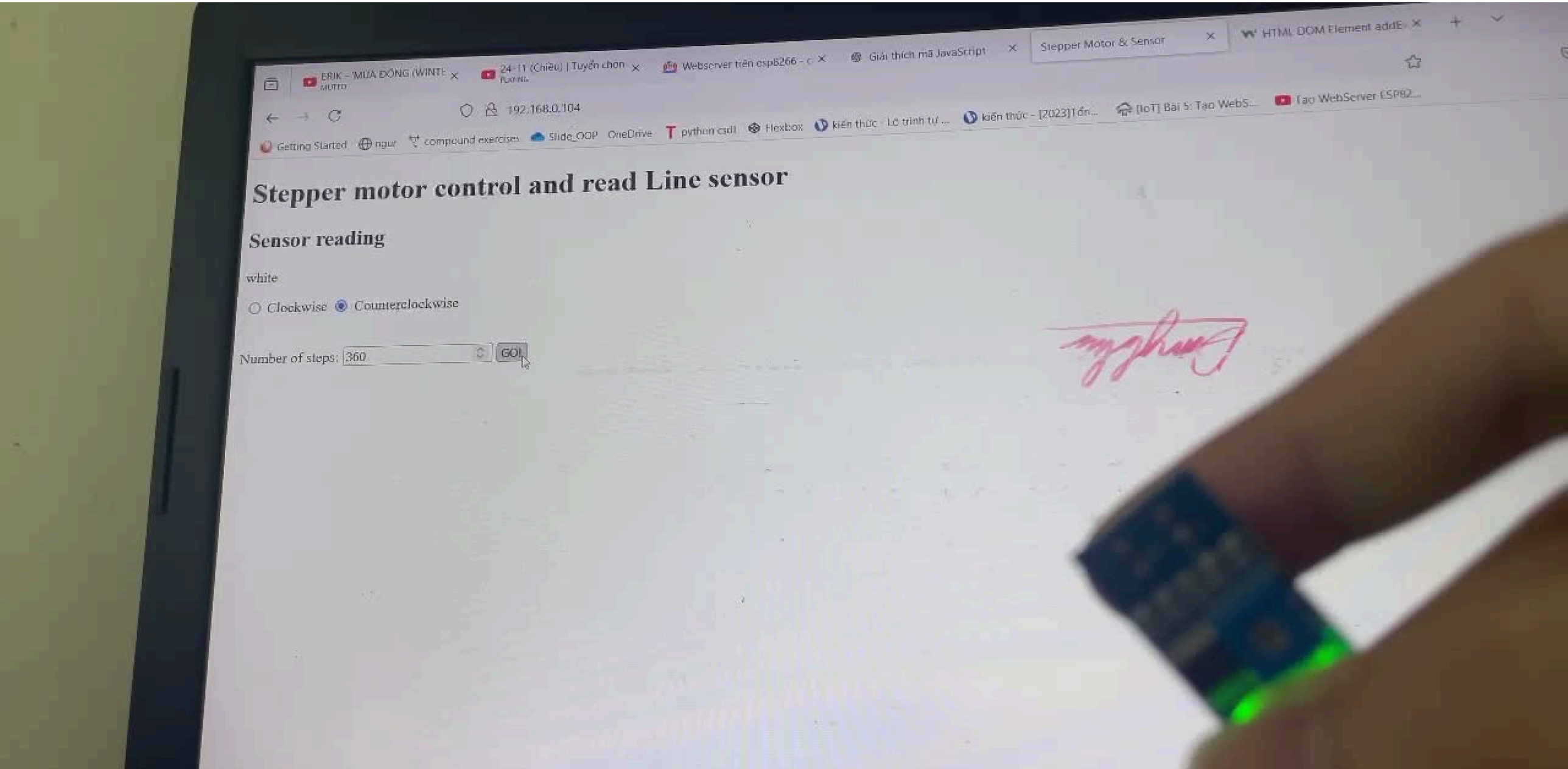
- Hàm server.on() tiếp theo xử lý dữ liệu trả về server từ web thông qua phương thức POST bằng hàm lambda như trên.
- Hàm này sẽ truy vấn qua từng tham số gửi về từ web, lấy giá trị nó ra và gán vào các biến dùng để điều khiển động cơ. Sau đó sẽ gửi lại nội dung HTML gốc cho trang, rồi báo rằng có request mới

Code

```
void loop()
{

  // Check if there was a new request and move the stepper accordingly
  unsigned long currentMillis = millis();
  if (newRequest && currentMillis - lastServoUpdate >= servoDelay) {
    lastServoUpdate = currentMillis;
    int stepCount = map(steps.toInt(), 0, 360, 0, 2048);

    if (direction == "CW") {
      for (int i = 0; i < stepCount; i++) {
        myStepper.step(-1);
        yield();
      }
    } else {
      for (int i = 0; i < stepCount; i++) {
        myStepper.step(1);
        yield();
      }
    }
    newRequest = false;
  }
}
```

- Ở hàm loop(), thực hiện việc chạy động cơ với dữ liệu đã nhận được từ web và xử lý xong.
- Hàm yield() để đảm bảo việc chạy động cơ ở không chiếm CPU quá nhiều tránh tình trạng lỗi.
- Chạy động cơ từng bước 1 để không bị quá tải cho module
- Clockwise(CW) là theo chiều kim đồng hồ

Kết quả


Stepper Motor & Sensor
HTML DOM Element addE
Giải thích mã JavaScript
Webserver trên esp8266 - c
192.168.0.104
Getting Started
python csdl
Flexbox
Slide_OOP
OneDrive
compound exercises
Stepper motor control and read Line sensor
Sensor reading
white
Clockwise
Counterclockwise
Number of steps:
360
GO!

Kết quả

4.2 Demo webserver điều khiển led

Lắp mạch

LED	Module WiFi
Chân dương	D5
Chân âm	GND

Code

```
//Khai báo thư viện
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>

//Khai báo thông tin wifi
const char* ssid = "Dang Khoa2"; //Tên wifi
const char* password = "dangkhoi"; //Mật khẩu wifi

//Khai báo đối tượng webserver, sử dụng port 80
ESP8266WebServer server(80);

//Khai báo chân LED
const int LED_PIN = D5;

//Khai báo biến chứa mã nguồn html dùng const char* R"html
const char html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
  <head>
    <meta name="viewport" content="width=device-width, initial-scale=1" charset="utf-8"
    <style>
      button {
        width:200px;
        height:100px;
```

Code

```
25          font-size:36px;
26        }
27      </style>
28    </head>
29    <body>
30      <h1>Webserver trên ESP8266</h1>
31      <p>Điều khiển LED qua giao diện web</p>
32      <p><a href="/led/on"><button>Bật LED</button></a></p>
33      <p><a href="/led/off"><button>Tắt LED</button></a></p>
34    </body>
35  </html>
36  )rawliteral";
37
38  //Hàm khởi tạo webserver
39  void setupServer() {
40    //Thiết lập chân LED là output
41    pinMode(LED_PIN, OUTPUT);
42    //Tắt LED
43    digitalWrite(LED_PIN, HIGH);
44
45    //Kết nối wifi
46    Serial.print("Connecting to ");
47    Serial.println(ssid);
48    WiFi.begin(ssid, password);
```

Code

```
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());

//Định nghĩa các hàm xử lý yêu cầu từ người dùng
//Nếu người dùng truy cập địa chỉ IP của ESP8266, trả về giao diện web để điều khiển LED
server.on("/", handleRoot);

//Nếu người dùng truy cập địa chỉ IP/led/on, bật LED và trả về thông báo
server.on("/led/on", handleLedOn);

//Nếu người dùng truy cập địa chỉ IP/led/off, tắt LED và trả về thông báo
server.on("/led/off", handleLedOff);

//Nếu người dùng truy cập địa chỉ IP không hợp lệ, trả về mã lỗi 404
server.onNotFound(handleNotFound);

//Bắt đầu webserver
server.begin();
```

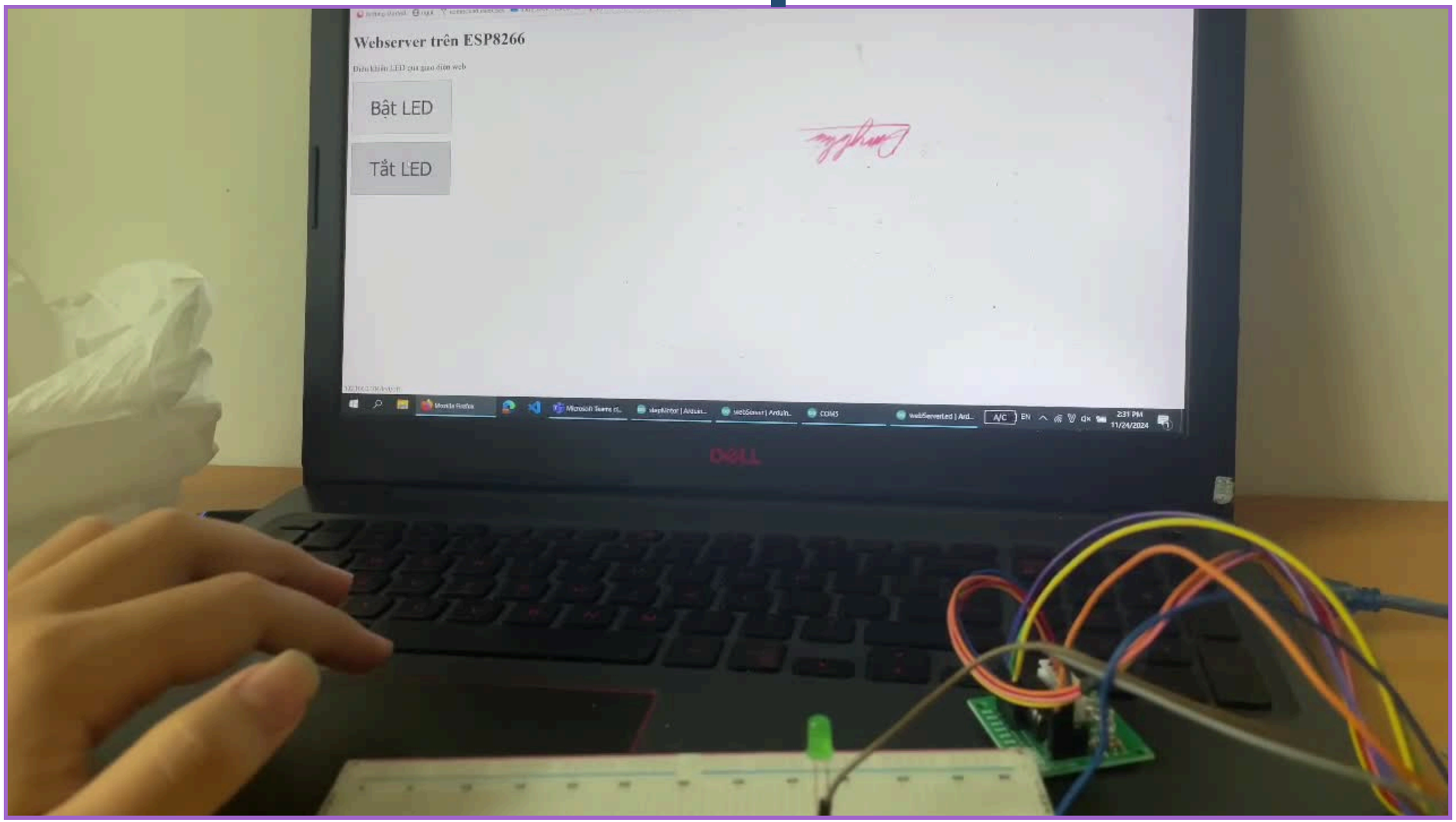
Code

```
  Serial.println("Webserver started");
}

//Hàm xử lý yêu cầu gốc
void handleRoot() {
  //Gửi biến chứa mã nguồn html cho người dùng
  server.send(200, "text/html", html);
}

//Hàm xử lý yêu cầu bật LED
void handleLedOn() {
  //Bật LED
  digitalWrite(LED_PIN, HIGH);
  //Gửi thông báo cho người dùng
  server.send(200, "text/html", html);
}

//Hàm xử lý yêu cầu tắt LED
void handleLedOff() {
  //Tắt LED
  digitalWrite(LED_PIN, LOW);
  //Gửi thông báo cho người dùng
  server.send(200, "text/html", html);
}
```

Code

```
//Hàm xử lý yêu cầu không hợp lệ
void handleNotFound() {
  //Gửi mã lỗi 404 cho người dùng
  server.send(404, "text/plain", "Khong tim thay trang Web");
}

//Hàm setup của Arduino IDE
void setup() {
  //Khởi tạo cổng nối tiếp
  Serial.begin(9600);
  //Khởi tạo webserver
  setupServer();
}

//Hàm loop của Arduino IDE
void loop() {
  //Xử lý các yêu cầu từ người dùng
  server.handleClient();
}
```

Kết quả



THANKS FOR LISTENING!

GOOD LUCK

:V